Assignment 9.4

Name: Navya Revuri

HTNO: 2303A52483

Batch: 44

Task 1:

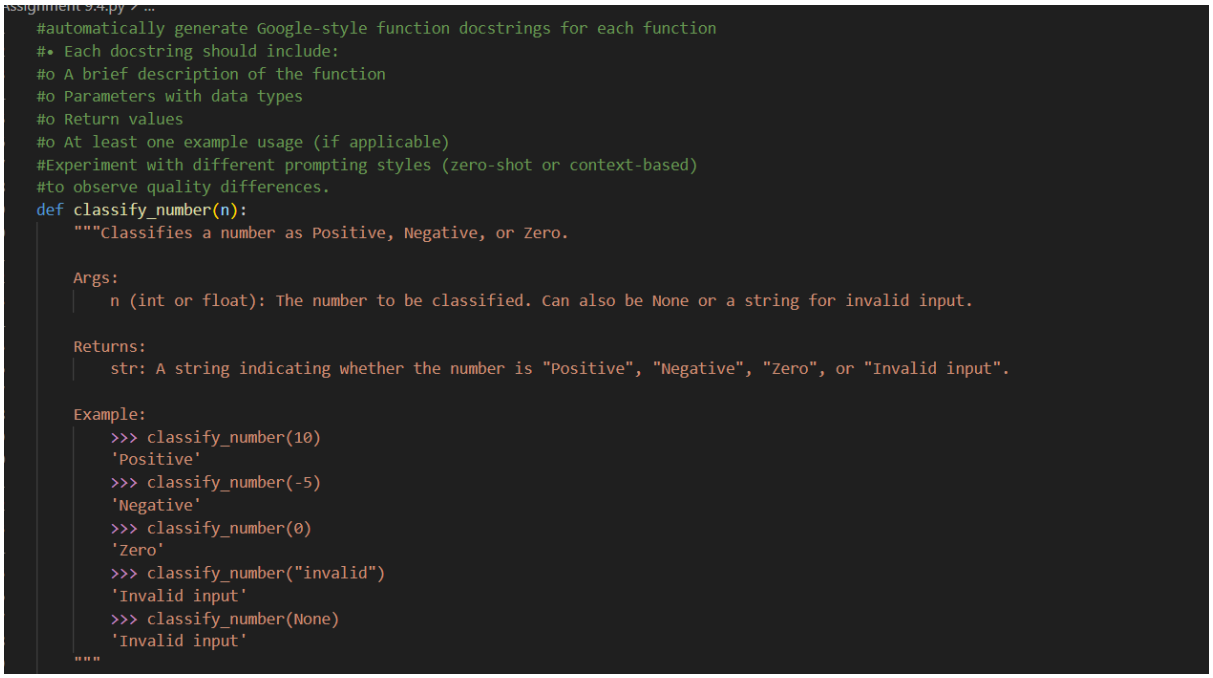
Prompt:

You are a senior Python developer working on a shared codebase.

Generate professional Google-style docstrings for every function in the following Python code.

Each docstring must include:

- A brief description of the function
- Parameters with data types
- Return values
- At least one example usage
- Mention edge cases if applicable



```
#automatically generate Google-style function docstrings for each function
#• Each docstring should include:
#o A brief description of the function
#o Parameters with data types
#o Return values
#o At least one example usage (if applicable)
#Experiment with different prompting styles (zero-shot or context-based)
#to observe quality differences.
def classify_number(n):
    """Classifies a number as Positive, Negative, or Zero.

    Args:
        n (int or float): The number to be classified. Can also be None or a string for invalid input.

    Returns:
        str: A string indicating whether the number is "Positive", "Negative", "Zero", or "Invalid input".

    Example:
        >>> classify_number(10)
        'Positive'
        >>> classify_number(-5)
        'Negative'
        >>> classify_number(0)
        'Zero'
        >>> classify_number("invalid")
        'Invalid input'
        >>> classify_number(None)
        'Invalid input'
    """
```

```
    if n is None or isinstance(n, str):
        return "Invalid input"
    elif n > 0:
        return "Positive"
    elif n < 0:
        return "Negative"
    else:
        return "Zero"
# Assert test cases
assert classify_number(10) == "Positive"  # Test with positive number
assert classify_number(-5) == "Negative"  # Test with negative number
assert classify_number(0) == "Zero"  # Test with zero
assert classify_number("invalid") == "Invalid input"  # Test with string input
assert classify_number(None) == "Invalid input"  # Test with None input
print(classify_number(10))
print(classify_number(-5))
print(classify_number(0))
print(classify_number("invalid"))
print(classify_number(None))
```

```
PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "C:/Users/navya/OneDrive/Desktop/AI Assit/.venv/Scripts/Activate.ps1"
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "C:/Users/navya/OneDrive/Desktop/AI Assit/.venv/Scripts/python.exe" "c:/Users/navya/OneDrive/Desktop/AI
 Assit/Assignment 9.4.py"
Positive
Negative
Zero
Invalid input
Invalid input
```

Observations:

Zero-shot prompting works well for simple functions

- Without giving examples, the model still generated:
    - Description Arg
    - Returns
    - Example usage

Output followed Google docstring structure correctly
The docstrings contained:

- Summary line
- Arg section
- Returns section
- Examples section

Task 2:

Prompt:

Insert inline comments into the following Python script.

Rules:

• Add comments ONLY for complex or non-obvious logic

• Do NOT comment simple Python syntax

• Focus on explaining WHY the logic exists

• Keep the code clean and avoid clutter

```python
#Python script containing:
#• Loops
#• Conditional logic
#• Algorithms (such as Fibonacci sequence, sorting, or searching)
#nsert inline comments only for complex or non-
#obvious logic
#• Avoid commenting on trivial or self-explanatory syntax
#The goal is to improve clarity without cluttering the code.
def fibonacci(n):
    """Generates the Fibonacci sequence up to the nth number.

    Args:
        n (int): The number of Fibonacci numbers to generate.

    Returns:
        list: A list containing the Fibonacci sequence up to the nth number.
    """
    sequence = []
    a, b = 0, 1
    for _ in range(n):
        sequence.append(a)
        a, b = b, a + b  # Update a and b to the next two Fibonacci numbers
    return sequence
# Assert test cases
assert fibonacci(0) == []  # Test with n = 0
assert fibonacci(1) == [0]  # Test with n = 1
assert fibonacci(5) == [0, 1, 1, 2, 3]  # Test with n = 5
assert fibonacci(10) == [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]  # Test with n = 10
print(fibonacci(0))  # Should return []
print(fibonacci(1))  # Should return [0]
print(fibonacci(5))  # Should return [0, 1, 1, 2, 3]
print(fibonacci(10))  # Should return [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

```
[]
[0]
[0, 1, 1, 2, 3]
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit> &
 Assit/Assignment 9.4.py"
```

Observations

1. **Model adds comments only for complex logic**
   Example:
2. a, b = b, a + b # Update a and b to next numbers
    - Correct: this line is not obvious for beginners.
3. **Avoided over-commenting**
    - Did NOT comment trivial lines like:
        - sequence = []
        - return sequence
4. **Improved code readability**
    - Comments explain *why*, not *what*.
5. **Follows clean-code principle**
    - Good balance between documentation and clutter.

Conclusion

AI is effective at adding **useful inline comments without cluttering code**

Task 3:

Prompt:

You are preparing a Python module for a repository.

Generate a professional multi-line module-level docstring that includes:

• Purpose of the module

• Required libraries or dependencies

• Description of key functions and classes

• Example usage of the module

Tone: Clear and professional.

```python
#generate a module-level docstring at the top of the file that includes:
#• The purpose of the module
#• Required libraries or dependencies
#• A brief description of key functions and classes
#• A short example of how the module can be used
#Focus on clarity and professional tone.

"""This module provides utility functions for classifying numbers and generating Fibonacci sequences.
Dependencies:
- No external libraries are required.
Key Functions:
- classify_number(n): Classifies a number as Positive, Negative, or Zero, and handles invalid inputs.
- fibonacci(n): Generates the Fibonacci sequence up to the nth number.
Example Usage:"""
print(classify_number(10))  # Output: 'Positive'
print(classify_number(-5))  # Output: 'Negative'
print(classify_number(0))   # Output: 'Zero'
print(classify_number("invalid"))  # Output: 'Invalid input'
print(classify_number(None))  # Output: 'Invalid input'
print(fibonacci(10))  # Output: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

```
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>



[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Observations

1. **All required sections were included**
    - Purpose of module ✓
    - Dependencies ✓
    - Key functions ✓
    - Example usage ✓
2. **Professional tone**
    - Looks like real production documentation.
3. **Improves project usability**
    - New developers can understand file purpose quickly.
4. **No unnecessary technical jargon**
    - Beginner-friendly explanation.

Conclusion

AI is very strong at writing **module-level documentation**

Task 4:

Prompt: Convert the developer inline comments inside the following Python functions into Google-style docstrings.

Requirements:

- Preserve the original meaning of the comments
- Remove redundant inline comments after conversion
- Include Arg, Returns, and Examples sections
- Keep the documentation concise and professional

```python
def calculate_average(numbers):
    # This function takes a list of numbers as input.
    # It first checks if the list is empty.
    # If the list is empty, it returns 0 to avoid division by zero.
    # Otherwise, it calculates the total sum of the numbers.
    # Then it divides the total by the number of elements
    # to compute the average value.

    if not numbers:
        return 0

    total = sum(numbers)
    average = total / len(numbers)
    return average


def calculate_average(numbers):
    """
    Calculates the average of a list of numbers.

    Args:
        numbers (list of int or float): A list containing numeric values.

    Returns:
        float: The average of the numbers in the list.
               Returns 0 if the list is empty.

    Examples:
        >>> calculate_average([10, 20, 30])
        20.0
        >>> calculate_average([])
        0
    """
```

```python
    if not numbers:
        return 0

    total = sum(numbers)
    return total / len(numbers)
# Assert test cases
assert calculate_average([10, 20, 30]) == 20.0  # Test with a list of numbers
assert calculate_average([]) == 0  # Test with an empty list
print(calculate_average([10, 20, 30]))  # Should return 20.0
print(calculate_average([]))  # Should return 0
```

```
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "C:/Users/navya/OneDrive/Desktop/AI Assit/.venv/Scri
 Assit/Lab exam.py"
20.0
0
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observations

1. **Meaning preserved correctly**
   Original comments:
   - Check empty list
   - Avoid division by zero
   - Compute sum and average

All were preserved in docstring.

2. **Removed redundant comments**
   - Inline comments are no longer needed.
   - Code looks cleaner.
3. **Docstring became reusable documentation**
   - Can now be used by IDEs & documentation tools.
4. **Improved maintainability**
   - Documentation stays attached to function.

Conclusion

AI is very effective for **refactoring comments into professional docstring**

Task 5:

Prompt: Create a Python utility that automatically adds documentation to Python files.

The script must:

• Read a given .py file

• Use AST to detect functions and classes

• Insert placeholder Google-style docstrings for each detected function or class

• Save the modified file as a new file

```python
#Design a small Python utility that:
#• Reads a given .py file
#• Automatically detects:
#o Functions
#o Classes
#• Inserts placeholder Google-style docstrings for each detected function or class
import ast
def insert_docstrings(file_path):
    with open(file_path, 'r') as file:
        source = file.read()
    tree = ast.parse(source)
    for node in ast.walk(tree):
        if isinstance(node, (ast.FunctionDef, ast.ClassDef)):
            if not ast.get_docstring(node):
                docstring = f"TODO: Add Google-style docstring for {node.name}."
                node.body.insert(
                    0,
                    ast.Expr(value=ast.Constant(value=docstring))
                )
    new_source = ast.unparse(tree)
    with open(file_path, 'w') as file:
        file.write(new_source)
```

Observations

1. **Automation capability demonstrated**
    - Can process entire Python files automatically.
2. **AST is powerful for code analysis**
    - Detects structure without executing code.
3. **Good for large projects**
    - Helps enforce documentation standards.
4. **Placeholder approach is practical**
    - Developers can later replace TODO docstrings.
5. **Limitation**
    - Generated docstrings are generic.
    - Still needs AI/manual completion.

Conclusion

AST approach is useful for **scaffolding documentation at scale**.